

Edge Detection

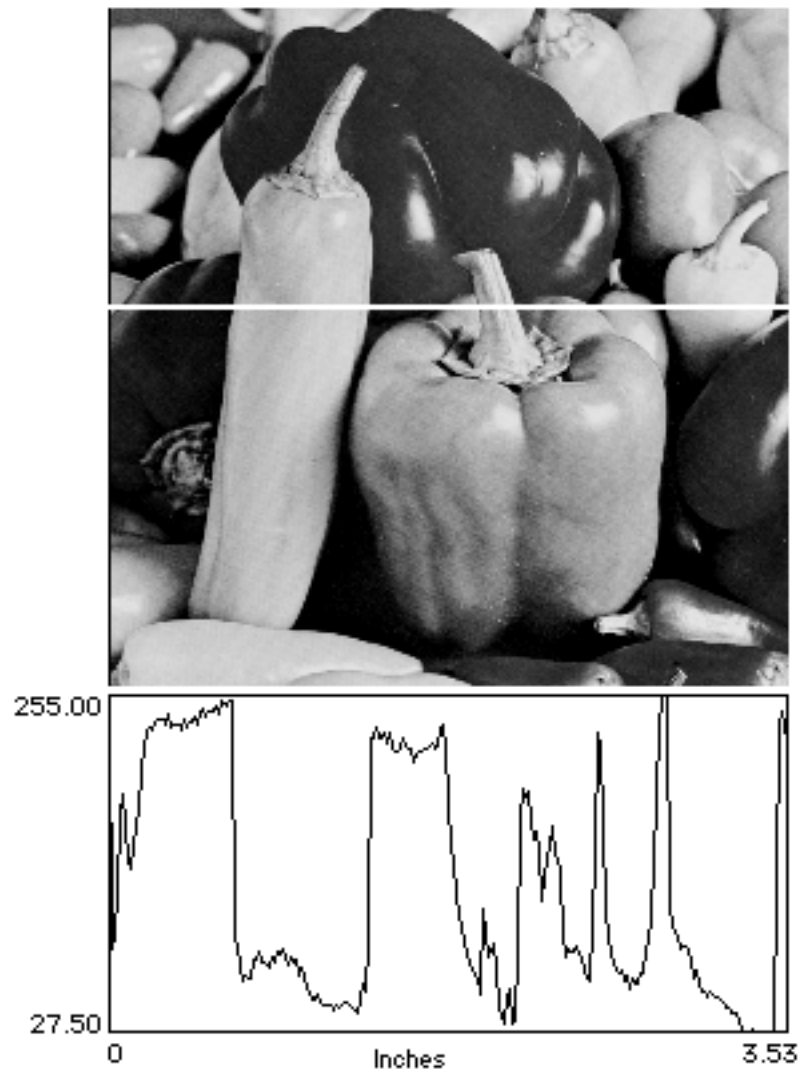
Brendan McCane
Department of Computer Science
University of Otago
Dunedin, New Zealand
mccane@cs.otago.ac.nz

1.0 Motivation

- Humans can recognise an enormous range of objects from just line images (eg cartoons).
- One theory (which can be backed up by physiological evidence) is that the human visual system (HVS) performs some sort of edge detection prior to recognition on colour or intensity images anyway.
- So it is reasonable to expect that some edge detection is required prior to image interpretation for automatic systems.
- Hence finding edges in an image is considered to be an important process in many artificial vision systems.

2.0 What is an edge?

→ A sharp change in intensity of an image!



→ How can we find edges in an image?

3.0 Image Gradients

→ The gradient of a 2D function can be defined as:

$$G[f(x, y)] = \begin{bmatrix} G_x \\ G_y \end{bmatrix} = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix}$$

$$= \begin{bmatrix} \lim_{\Delta x \rightarrow 0} \frac{f(x + \Delta x, y) - f(x, y)}{\Delta x} \\ \lim_{\Delta y \rightarrow 0} \frac{f(x, y + \Delta y) - f(x, y)}{\Delta y} \end{bmatrix}$$

→ This is a vector quantity!

→ Of course, we are dealing with discrete images not continuous functions, so we can really only approximate the gradient of an image.

→ How can we do this?

→ The simplest way is to set $\Delta x = \Delta y = 1$ which results in the following formula:

$$G[f[i, j]] = \begin{bmatrix} f[i + 1, j] - f[i, j] \\ f[i, j + 1] - f[i, j] \end{bmatrix}$$

→ This formula can be represented using the following convolution masks:

$$G_x = \begin{bmatrix} -1 & 1 \end{bmatrix}$$

$$G_y = \begin{bmatrix} -1 \\ 1 \end{bmatrix}$$

→ Other Approximations are also possible including:

→ Roberts cross operator:

$$G_x = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

$$G_y = \begin{bmatrix} 0 & -1 \\ -1 & 0 \end{bmatrix}$$

→ Prewitt operator:

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$$

$$G_y = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$$

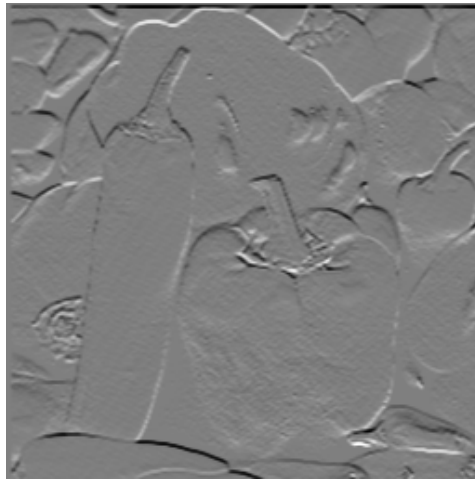
→ Sobel operator:

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

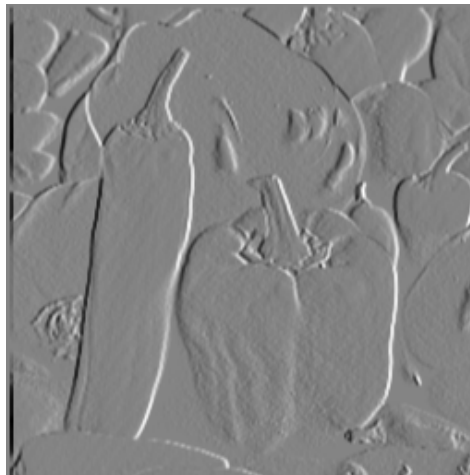
$$G_y = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$

→ An example using Sobel:

Y gradient



X gradient



4.0 Numerical Differentiation

4.1 The First Derivative

→ The previous filter masks are examples of numerical differentiation which we should look at more closely.

→ Recall that the Taylor series expansion of a function about $x = a$ is:

$$f(x) = f(a) + f'(a)(x-a) + \frac{f''(a)(x-a)^2}{2!} + \frac{f'''(a)(x-a)^3}{3!} + \dots \quad (\text{EQ 1})$$

→ So, if we expand $f(x + \Delta x)$ about x , we get:

$$f(x + \Delta x) = f(x) + \Delta x f'(x) + \frac{(\Delta x)^2}{2!} f''(x) + \dots \quad (\text{EQ 2})$$

→ Rearranging we get:

$$\frac{f(x + \Delta x) - f(x)}{\Delta x} = f'(x) + \frac{\Delta x}{2!} f''(x) + \dots \quad (\text{EQ 3})$$

→ So using our simple approximation for the derivative results in an analytic error proportional to Δx .

→ For no particular reason, let's do the following:

$$f(x - \Delta x) = f(x) - \Delta x f'(x) + \frac{(\Delta x)^2}{2!} f''(x) - \dots \quad (\text{EQ 4})$$

→ Now, subtract Equation 4, on page 6 from Equation 2, on page 6:

$$f(x + \Delta x) - f(x - \Delta x) = 2\Delta x f'(x) + \frac{2(\Delta x)^3}{3!} f'''(x) + \dots \quad (\text{EQ 5})$$

→ Rearranging:

$$\frac{f(x + \Delta x) - f(x - \Delta x)}{2\Delta x} = f'(x) + \frac{(\Delta x)^2}{3!} f'''(x) + \dots \quad (\text{EQ 6})$$

→ Of course, when we're talking about images $\Delta x = 1$, so what does this mean for the error?

→ What if we estimate the error between two neighbouring pixels?

→ Equation 6, on page 6 is called the two point central difference approximation.

→ Other approximations using more terms are also possible

4.2 The Second Derivative

→ We can also approximate the second derivative using a similar procedure:

$$\begin{aligned} f''(x) &= \frac{f'(x + \Delta x) - f'(x)}{\Delta x} \\ &= \frac{f(x + 2\Delta x) - 2f(x + \Delta x) + f(x)}{\Delta x^2} \end{aligned} \quad (\text{EQ 7})$$

→ Giving an error of $O(\Delta x)$.

→ We can similarly use the 2 point central difference to give a better approximation of the second derivative.

5.0 Edgels

→ If we have the derivative in the x and y directions, then we have two orthogonal measures which gives us information about the derivative in all directions if we assume the surface is well behaved (ie differentiable).

→ So for each pixel we have two derivative values, and this is commonly called an edgel.

→ Edgels are more commonly stored as a magnitude and a direction.

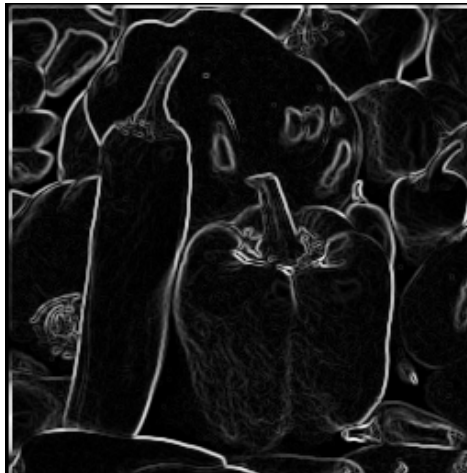
6.0 Gradient Magnitude

→ All well and good, but how do we use this information?

→ Often it is more useful to look at the gradient magnitude of an image:

$$|G| = \sqrt{G_x^2 + G_y^2} \quad (\text{EQ 8})$$

→ Which looks like this:



→ Notice how the edges are now quite distinct.

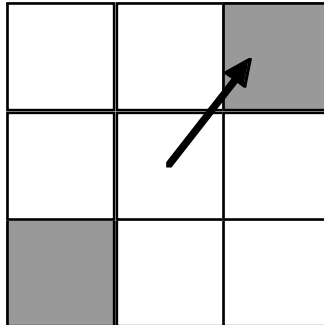
→ Edge pixels can now be identified as those pixels whose value is above a certain threshold. This process is called thresholding:



7.0 Non-maximal Suppression

- By defining an edge pixel as one with a large change in intensity we often get edges greater than one pixel thick.
- It is often more convenient to restrict edges to exactly one pixel.
- We can apply some sort of thinning technique, or alternatively, define edge pixels as those which have a local gradient maxima (ie only those pixels lying along the ridge of the intensity surface).
- To find these pixels, we need to perform non-maxima suppression.

- The gradient of a 2d function indicates the direction in which the function is changing most rapidly, so to determine if a given point in the gradient magnitude image is a maximum, we need to check surrounding points in the direction of the gradient.
- Ie check these points in the gradient magnitude image:



- The neighbouring points can be chosen as the closest in the positive and negative direction of the gradient, or by interpolating between the four closest neighbours (which should give more accurate results).
- And here's the result:



8.0 Hysteresis Thresholding

- We can often do better than selecting one threshold by selecting two - an upper and a lower threshold.
- The rule for outputting a pixel is then that it is either greater than the upper threshold, or greater than the lower threshold and connected to a pixel who is greater than the upper threshold.

- This technique is called hysteresis thresholding and will generally give better connectivity without introducing many spurious edges:



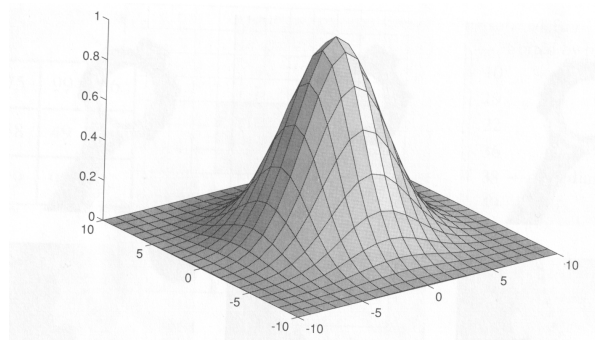
9.0 Gaussian Smoothing

- The very nature of the image formation process produces noise in an image.
- This can cause even greater problems of error for numerical differentiation.
- Problems with noise are typically handled by smoothing images (eg mean or box filter).
- One of the most common smoothing filters is the 2d Gaussian filter:

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{\frac{-((x-x_0)^2 + (y-y_0)^2)}{2\sigma^2}} \quad (\text{EQ 9})$$

where σ is the width or standard deviation of the filter, and (x_0, y_0) is the centre or mean of the filter.

- Here is what a Gaussian filter looks like:



9.1 Features of the Gaussian Filter

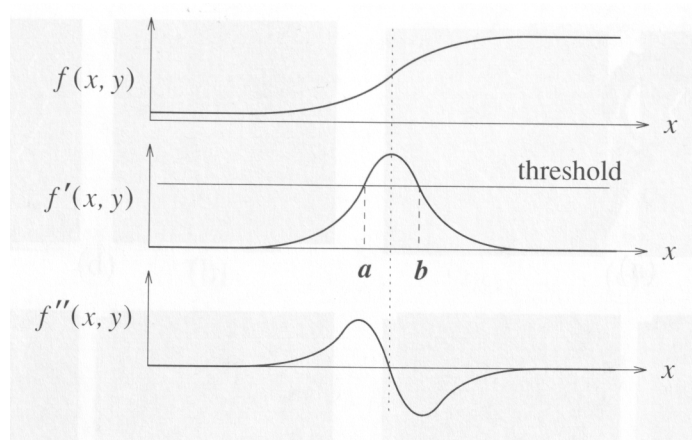
- 1.0 It's a linear, shift invariant filter and therefore can be implemented by convolution.
- 2.0 Rotationally symmetric.
- 3.0 The Fourier transform of a Gaussian is another Gaussian.
- 4.0 Is separable. That is, the 2D Gaussian can be implemented using 2 orthogonal 1D Gaussians.
- 5.0 Approaches 0 at more than 3 standard deviations from the mean and therefore can be approximated by a filter of local extent.
- 6.0 Provides a good trade off between localisation of edges and minimisation of Gaussian noise in an image.

10.0 Canny's Edge Detector

- Canny introduced many of the above techniques in developing his edge detector which is still one of the most practical and commonly used techniques today.
- It consists of the following steps:
 - 1.0 Smooth an image with a Gaussian filter.
 - 2.0 Calculate the gradient (magnitude and orientation) of the smoothed image.
 - 3.0 Apply non-maximal suppression.
 - 4.0 Perform hysteresis thresholding.
- The thresholds to be used for hysteresis thresholding can be calculated based on noise estimates in the image, but often they are set by hand.
- Canny also suggested using operators with multiple widths to detect edges. If any of the operators reliably detect an edge pixel, then that pixel is marked as an edge pixel. If multiple operators detect an edge pixel, then the smallest operator is chosen since it has the best localisation.

11.0 Second Derivative Operators

→ Finding the maximum of the derivative is equivalent to finding the zero-crossings of the second derivative:



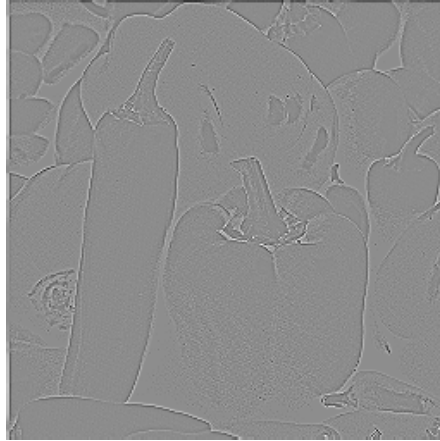
11.1 The Laplacian

→ When we go to the second derivatives in 2D we get 3 potential operators:

$$\frac{\partial^2 f}{\partial x^2}, \quad \frac{\partial^2 f}{\partial x \partial y}, \quad \frac{\partial^2 f}{\partial y^2} \quad (\text{EQ 10})$$

→ But it is common to use the Laplacian operator as a scalar 2nd derivative in more than one dimension:

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2} \quad (\text{EQ 11})$$



→ And the zero-crossings are:



- Clearly, there's too many, so we can threshold before looking for zero-crossings to produce something like this:



- What's the first thing you notice about this image compared to the first derivative operator? Why would that be true?

11.2 The Laplacian of Gaussian

- Clearly, we need to do something about noise in the image, and to do that we can smooth with a Gaussian:

$$h(x, y) = \nabla^2[g(x, y) \otimes f(x, y)] \quad (\text{EQ 12})$$

- In other words, smooth the image with a Gaussian filter, and then extract zero crossings of the Laplacian.

- Using the derivative property of convolution we get:

$$h(x, y) = [\nabla^2 g(x, y)] \otimes f(x, y) \quad (\text{EQ 13})$$

- So, we can actually convolve an image with the Laplacian of a Gaussian to get the effects of smoothing and extract edges in one hit (pretty cool huh?).

→ The LoG filter is also called the Mexican hat operator:

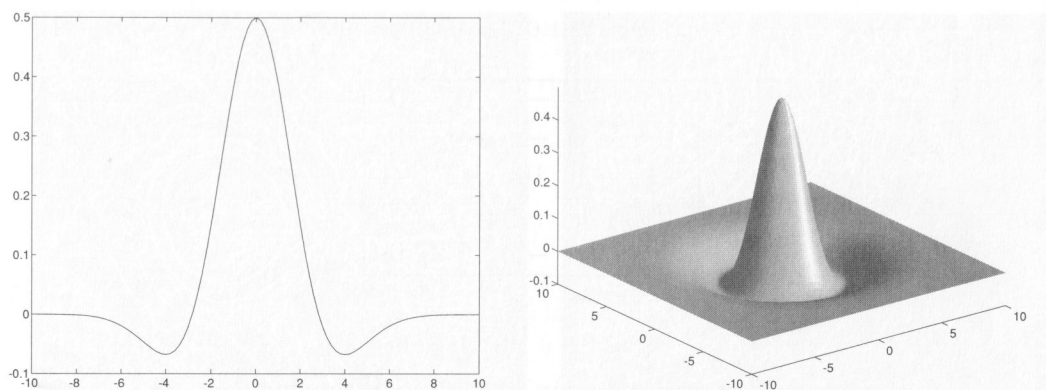


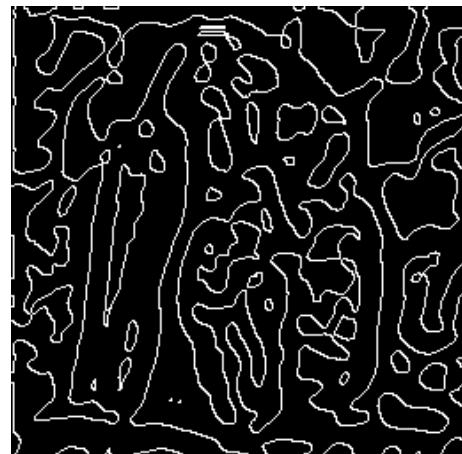
Figure 5.11: The *inverted* Laplacian of Gaussian function, $\sigma = 2$, in one and two dimensions.

→ Zero-crossings of a LoG filtered image tend to produce closed contours which can be useful in some situations - of course, this means that there doesn't have to be a strong edge in all positions along the contour.

→ And this is what it looks like (without thresholding):



sigma = 1.0



sigma = 4.0

→ We can alter the number of extracted edges either by changing the width of the Gaussian filter or by introducing some sort of thresholding technique (although this may not produce closed contours).

- Historically, the LoG operator came before Canny and there is evidence to support similar operators in the human visual system. However, first derivative operators are generally preferred due to their better noise-handling abilities (but they are typically more expensive to calculate).

12.0 Exercises

- 1.0 Derive the central difference approximation to the second derivative using the two-point central difference approximation to the first derivative.
- 2.0 Derive the LoG operator.
- 3.0 Try out Prewitt's and Sobel's operators for calculating the first derivative. Use these to form the gradient magnitude. Test your results on a few images and see what you get.
- 4.0 In a practical sense, how can we reduce the error of calculating the gradient of an image using just a two-point central difference?
- 5.0 Design an algorithm to find the zero-crossings in an image.

13.0 Bibliography

- [1]J. Shen, "on Multi-Edge Detection," Graphical Models and Image Processing, vol. 58, pp. 101-114, 1996.
- [2]J. F. Canny, "A computational approach to edge detection," IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 8, pp. 679-698, 1986.
- [3]R. Deriche, "Optimal edge detection using recursive filtering," in Proceedings of the First International Conference on Computer Vision. London, June 1987.
- [4]D. Marr and E. Hildreth, "Theory of edge detection," Proceedings Royal Society of London, vol. B 207, pp. 187-217, 1980.
- [5]M. C. Morrone, A. Navangione, and D. Burr, "An adaptive approach to scale selection for line and edge detection," Pattern Recognition Letters, vol. 16, pp. 667-677, 1995.
- [6]P. Perona and J. Malik, "Scale-Space and Edge Detection Using Anisotropic Diffusion.," IEEE Transactions of Pattern Analysis and Machine Intelligence, vol. 12, pp. 629 - 639, July 1990.
- [7]J. Shen and S. Castan, "An optimal linear operator for edge detection," presented at Proceedings IEEE Computer Society Conference on Computer Vision and Pattern Recognition '86, 1986.
- [8]J. Shen and S. Castan, "Edge detection based on multi-edge models," presented at Proceedings of The International Society for Optical Engineering (SPIE) '87, 1987.
- [9]V. Torre and T. A. Poggio, "On edge detection," IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 8, pp. 147-163, March, 1986.